

Gate-Logged Self-Auditing Control for Adaptive Compute, Routing, Memory, and Trajectory Management

Sohan Poudel
forreaserachonlyso@gmail.com
ORCID: 0009-0004-5762-3898

May 30, 2026

Transparency and Provenance Note

This document is an early research draft built around a simple personal idea: internal controller decisions should be logged before and after action so that compute allocation, routing, memory use, and stop/restart behavior can be audited and trained directly. The author does not claim prior expertise in all areas of sparse routing, memory systems, calibration, or neural-controller theory, and presents this work as an exploratory experiment rather than a settled scientific result.

Parts of the brainstorming, code scaffolding, analysis, and manuscript drafting were produced with the assistance of large language models, including ChatGPT, Gemini, and Grok, and were then selected, checked, and edited by the author. The main purpose of including this note is transparency: the underlying idea came from the author, while LLMs were used as exploratory and writing tools to help turn that idea into runnable experiments and a readable draft.

For additional transparency, the author intends to provide the originating LLM conversations and experiment notebook used for the experiments. The following are the links:

- Originating ChatGPT conversation:
<https://chatgpt.com/share/6a1ade6f-7220-83a2-8204-144702dc7da8>
- Originating Gemini conversation:
<https://gemini.google.com/share/179f3ecf1512>
- Originating Grok conversation:
https://grok.com/share/bGVnYWN5LWNvcHk_33b6623c-5fd0-45fa-a23e-c314fa57f096
- Experiment notebook / Colab:
https://colab.research.google.com/drive/1e70UQtRyokcSQc_UhL3BGdLFeJVARH5K?usp=sharing

This manuscript should not be interpreted as a peer-reviewed claim of novelty, priority, or superior performance. Readers should independently verify experiments, compare against prior work, and treat the present results as an initial proof-of-concept study.

Abstract

We study a four-gate self-auditing controller for adaptive compute allocation, route selection, memory retrieval, and trajectory control. For each decision, the controller records an entry note

before acting and an exit note after acting, then learns from task loss, prediction–outcome alignment, and counterfactual regret. The goal is not merely to explain final outputs, but to make internal control decisions measurable, auditable, and trainable.

The experiments support a cautious view. In a synthetic setting, the controller learns the intended gate structure and sharply reduces regret. In small real-data routing tests, however, unconstrained hard gate choice is less reliable: it can collapse to cheap gates, fail to beat a strong single expert, or achieve high task accuracy while still missing the gate oracle. On sklearn digits, the strongest simple result is a constrained soft budget: a 40% Memory allocation reaches mean task accuracy 0.976, above the best single expert at 0.961. A small MasterFabric integration reaches 0.9694 digits accuracy, but only 0.6000 gate accuracy, again separating task success from gate-target alignment.

Across CIFAR-10, IMDB, AG News, Breast Cancer, FashionMNIST, and 20 Newsgroups, no single routing policy dominates. Memory-heavy mixtures, no-memory mixtures, disagreement-aware mixtures, fixed rule priors, note-only routers, and note-plus-feature routers each win in different settings. Later stress tests narrow the claims further: running memory sometimes gives small gains, but often matches shuffled or noisy controls; router confidence helps selective routing, but calibration is dataset-dependent and weak on text. Overall, gate-logged control is best understood as an experimental framework for auditing and improving internal decisions. Stronger claims require controlled evidence for stable budget allocation, ordered gate paths, executable rule priors, calibrated fallback decisions, and memory mechanisms that consistently beat explicit control baselines.

1 Introduction

Modern learned systems are commonly evaluated by their final outputs: an answer is correct or incorrect, a plan succeeds or fails, and a prediction receives a scalar reward or loss. However, many failures arise earlier, inside the system’s control process. A model may spend too much compute on easy inputs, choose the wrong processing route, retrieve irrelevant memory, continue a failing trajectory, or stop before sufficient evidence has been gathered. In these cases, the final output reveals that something went wrong, but it provides only a weak signal about which internal decision caused the failure.

This paper studies a standalone *Gatekeeper Controller* that learns to make four kinds of internal decisions. For each input and state, the controller must choose whether to allocate compute, select a route, retrieve memory, or stop and restart a trajectory. Before acting, it records an entry note explaining why the gate was selected, what outcome it expects, how confident it is, and what cost it predicts. After acting, it records an exit note describing what actually happened, whether the decision helped, how much gain or cost occurred, and whether the original prediction was accurate.

The main claim is not that the system writes notes. The contribution is a self-auditing controller that learns an internal decision policy from pre-action justification, post-action outcome, and counterfactual comparison. The controller is therefore judged by how well it decided, not only by whether the final answer was correct.

This separation is deliberate. The four-gate controller should first be tested by itself, before it is attached to a larger modular architecture. The first question is whether gate-logged control works as an independent mechanism. Only after that is established should it be used as the control layer for a broader multi-paper system.

Run note. A toy controller trained on 5,000 synthetic examples improved GateAcc from 0.520 to 0.925 and regret from 0.141 to 0.003; a single-seed digits sanity test reached 0.906 task accuracy,

a multi-seed multi-factor digits test found the best mean setting at 0.853 task accuracy, and a 10-pass episodic test improved task accuracy over a static controller by 0.029. A hybrid rule-plus-learned controller then reached 0.962 mean task accuracy, while a note-order test showed that extra read/write cycles can collapse to one gate if not regularized. A budget sweep then found that a 40% Memory allocation produced the best mean task accuracy at 0.976, and a connected-room path sweep found Stop/Restart→Memory best at 0.970. A combined large sweep found Memory-40 at full scale to be the best tested setting, with 0.976 mean accuracy. A multi-dataset sweep on CIFAR-10, IMDB, and AG News found Memory-mix strongest among the tested strategies, but later weakness-hunt and cross-modal sweeps showed that no-memory, disagreement-aware, and rule-prior mixtures can be stronger depending on modality and expert agreement.

2 Four-Gate Control Problem

At each step t , the controller receives an input state

$$x_t = \{\text{task input, current state, history, memory summary, budget estimate}\}. \quad (1)$$

Some components may be optional depending on the environment. The controller then chooses a gate from

$$\mathcal{G} = \{\text{Compute, Route, Memory, Stop/Restart}\}. \quad (2)$$

The policy is written as

$$\pi_{\theta}(g_t \mid x_t), \quad g_t \in \mathcal{G}, \quad (3)$$

where θ denotes the controller parameters. Unlike a standard routing policy, this policy also emits an auditable record of its decision.

Run note. The toy gates used synthetic hidden factors, while the digits sanity test used four real experts: MLP as Compute, PCA–logistic regression as Route, KNN as Memory, and decision tree as Stop/Restart. This shows that each gate can be instantiated either as a controlled operation or as a real expert path.

2.1 Gate 1: Compute

The compute gate decides whether the system should spend additional computation or exit early. Possible operations include using a heavier model path, activating more layers, performing extra refinement, or stopping when confidence is already sufficient. Useful signals include uncertainty, task complexity, predicted gain, and predicted cost. Evaluation signals include accuracy improvement, FLOPs, latency, and efficiency per token.

2.2 Gate 2: Route

The route gate selects an internal processing path. Possible operations include a sparse route, dense route, directional route, expert module, or specialized subnetwork. Useful signals include task type, structural pattern, and expected route fit. Evaluation signals include route success, sparsity efficiency, and regret relative to alternative routes.

2.3 Gate 3: Memory

The memory gate decides whether retrieval is needed and, if so, what type of retrieval to perform. Possible operations include no retrieval, short-context retrieval, long-context retrieval, or top- k memory lookup. Useful signals include context drift, ambiguity, repeated patterns, and missing details. Evaluation signals include retrieval usefulness, noise introduced by retrieval, and performance improvement after retrieval.

2.4 Gate 4: Stop/Restart

The stop/restart gate controls trajectory continuation. Possible operations include continuing forward, halting and returning an answer, overwriting a bad intermediate state, rolling back, or restarting the trajectory. Useful signals include instability, contradiction, lack of progress, and boundary violations. Evaluation signals include whether restart improved performance, whether early stopping was correct, and whether continuation wasted compute.

3 Gate-Logged Audit Records

The controller produces two structured records for every decision: an entry note before action and an exit note after action. These records may be represented as text, JSON, typed fields, or a hybrid format. The important property is that they expose a prediction that can later be scored.

3.1 Entry Note

The entry note records the controller’s pre-action belief as compact fields: `selected_gate`, `reason`, `expected_gain`, `expected_cost`, `confidence`, and `risk`.

For example, a memory decision may record: *selected gate: memory; reason: context drift is high; expected gain: retrieval will reduce error; expected cost: moderate; confidence: high; risk: retrieved memory may add noise.*

3.2 Exit Note

The exit note records the post-action result as compact fields: `actual_gain`, `actual_cost`, `success`, `prediction_error`, and `lessons`.

The audit record therefore contains what the controller thought would happen and what actually happened. This enables direct training of decision calibration.

Run note. Entry notes used selected gate, predicted gain, confidence, and reason; exit notes used actual gain, best gate, regret, and prediction error. The inspected examples had zero regret and small prediction errors, so audit fields should be structured enough to score directly.

4 Learning Objective

The controller learns from three sources: task success, gate alignment, and counterfactual regret.

4.1 Task Loss

The ordinary task loss is denoted

$$L_{\text{task}}. \quad (4)$$

Depending on the environment, this may be classification error, sequence loss, planning error, reconstruction loss, or another task-specific objective.

4.2 Gate Alignment Loss

Gate alignment measures whether the entry note predicted the real result. If the controller predicts gain $\hat{g}_t$ and observes actual gain g_t , a simple alignment loss is

$$L_{\text{align}} = (\hat{g}_t - g_t)^2. \quad (5)$$

The loss can be expanded to include cost and usefulness:

$$L_{\text{align}} = \alpha(\hat{g}_t - g_t)^2 + \beta(\hat{c}_t - c_t)^2 + \gamma(\hat{u}_t - u_t)^2, \quad (6)$$

where c_t is actual cost, u_t is a usefulness or success signal, and α, β, γ weight the components.

4.3 Counterfactual Regret

Counterfactual regret penalizes the controller when a different gate would have achieved a better score. Let $S(x_t, k)$ be the measured or estimated score from choosing gate k in state x_t . Regret is

$$R_t = \max_{k \in \mathcal{G}} S(x_t, k) - S(x_t, g_t). \quad (7)$$

This term asks how much worse the chosen gate was than the best available alternative.

4.4 Total Objective

The complete training objective is

$$L_{\text{total}} = L_{\text{task}} + \lambda L_{\text{align}} + \mu R_t, \quad (8)$$

where λ controls the weight of decision-alignment supervision and μ controls the weight of counterfactual regret.

Run note. With cross-entropy, gain MSE, and regret all active ($\lambda = \mu = 1$), RMSE fell from 0.258 to 0.050 and regret from 0.141 to 0.003, showing that predicting alternative gate values is useful beyond classifying the best gate.

5 Training Loop

The training process is a repeated audit cycle: observe x_t , write the entry note, execute the selected gate, measure outcome and cost, write the exit note, compare prediction against result and alternatives, and update L_{total} .

This differs from ordinary reinforcement learning. In a standard loop, an action is taken, a reward is observed, and the policy is updated. Here, the controller also commits to a reason and a prediction before acting. The reason and prediction then become supervised objects that can be compared with the observed result.

Run note. The loop used batches of 128 for 25 Adam epochs; test loss fell from 1.4232 to 0.1844, showing that the audit cycle can be tested with a simple supervised-and-regret prototype.

6 Experimental Design

The four-gate system should first be tested as a standalone controller. The goal is to determine whether gate-logged auditing improves internal control before introducing a larger modular stack.

6.1 Phase A: Synthetic Sandbox

A synthetic environment should assign known tradeoffs to each gate. For example, extra compute helps on hard inputs but wastes cost on easy inputs; memory helps only when information is missing; restart helps only when the state is unstable; and route choice matters according to input type. This setting tests whether the controller learns sensible gate choices when the ground-truth tradeoffs are known.

6.2 Phase B: Benchmark Tasks

The controller should then be evaluated on tasks where internal decisions matter, including reasoning, multi-step planning, long-context question answering, noisy inputs, hidden memory dependence, and settings where early stopping is sometimes beneficial.

6.3 Phase C: Baselines

Recommended baselines include no gate controller, random gate choice, gate choice without notes, notes without alignment loss, an ordinary reinforcement-learning router, and a reflection-only agent.

6.4 Phase D: Ablations

Useful ablations remove one ingredient at a time: no entry note, no exit note, no regret loss, no memory gate, and no stop/restart gate. These experiments identify which components actually contribute to performance.

Run note. The prototype completes Phase A only: performance rose far above random chance (0.25) and regret approached zero, so the next step is baselines and ablations against random gates, no notes, no alignment, and no regret.

6.5 Phase E: Small Real-Data Sanity Test

As real-data checks, we used the sklearn digits dataset with four experts: Compute = MLP, Route = PCA plus logistic regression, Memory = KNN, and Stop/Restart = decision tree. In the single-seed version, the controller learned the best cost-adjusted expert, $P_j(\text{true label} \mid x_i) - \text{cost}_j$, and reached $\text{TestTaskAcc} = 0.906$, $\text{GateAcc} = 0.686$, $\text{RMSE} = 0.277$, and $\text{Regret} = 0.097$. In the stronger multi-factor version, utility combined correctness, true-label probability, prediction margin, entropy, and cost across five seeds and three utility settings. The best mean task accuracy came from the balanced setting: $\text{TaskAcc} = 0.8533 \pm 0.0048$, $\text{GateAcc} = 0.7717 \pm 0.0110$, $\text{Regret} = 0.0921 \pm 0.0035$, $\text{RMSE} = 0.2326 \pm 0.0024$, and $\text{AvgCost} = 0.0123 \pm 0.0003$. This remains below the best single expert Memory at 0.961 and the oracle ceiling near 0.992, but it shows non-random gate learning under real-data, multi-objective utility. We also tested a 10-pass episodic controller with history; compared with a static one-pass controller, it raised task accuracy from 0.856 to 0.885, increased gate diversity, and exposed a tradeoff because regret rose slightly from 0.090 to 0.098. Finally, a multi-rule hybrid test compared pure learned, rule-only, rule-regularized, and hybrid controllers; the best configuration was hybrid-accuracy with mean $\text{TaskAcc} = 0.9622 \pm 0.0014$, slightly above the best single expert Memory at 0.9611, but with low gate-target accuracy because it mostly chose Memory.

7 Metrics

The paper should report both final-task metrics and internal-decision metrics. Table 1 summarizes the main quantities.

Run note. The synthetic final metrics were $\text{GateAcc} = 0.925$, $\text{ValueTopAcc} = 0.928$, $\text{RMSE} = 0.050$, and $\text{Regret} = 0.003$; the single-seed digits metrics were $\text{TaskAcc} = 0.906$, $\text{GateAcc} = 0.686$, $\text{RMSE} = 0.277$, and $\text{Regret} = 0.097$; the best multi-seed digits setting was balanced with mean $\text{TaskAcc} = 0.853$, $\text{GateAcc} = 0.772$, and $\text{Regret} = 0.092$; the episodic test improved task accuracy from 0.856 to 0.885 but increased regret from 0.090 to 0.098; the hybrid accuracy rule reached mean $\text{TaskAcc} = 0.962$, $\text{AvgCost} = 0.048$, and $\text{Regret} = 0.101$; the note-order test found no gain from two-pass note cycles, with all schedules at $\text{TaskAcc} = 0.828$ and $\text{Regret} = 0.136$; the budget sweep found Memory-40 best with $\text{TaskAcc} = 0.976$, above the best single expert at 0.961; the best two-gate connected path was Stop/Restart→Memory with $\text{TaskAcc} = 0.970$; the combined large sweep confirmed memory_mix at scale 1.00 as best with $\text{TaskAcc} = 0.976$ and $\text{Regret} = 0.098$; the multi-dataset sweep found Memory-mix best on CIFAR-10, IMDB, and AG News, while later cross-modal sweeps found no-memory or rule-prior mixtures best on several tabular, image, and text settings.

Metric	Purpose
Final task accuracy	Measures end performance
Compute saved	Measures efficiency of compute decisions
Memory usefulness	Measures whether retrieval helped
Route selection accuracy	Measures path-choice quality
Stop/restart correctness	Measures trajectory-control quality
Gate alignment score	Measures prediction calibration
Counterfactual regret	Measures missed better gate choices
Efficiency per FLOP	Measures cost-normalized performance
Efficiency per token	Measures token-normalized performance
Learning curve	Measures improvement over training

Table 1: Suggested metrics for evaluating gate-logged control.

8 Experimental Program

The proposed controller is best understood as a mechanism for learning internal decision quality. It does not claim to solve all routing, memory, compute, or trajectory-control problems. Instead, it provides a structured way to ask whether the system’s own reasons predicted its outcomes and whether its selected gate was better than available alternatives.

The experiments below therefore move from controlled mechanism tests to increasingly difficult real-data routing tests. The synthetic prototype asks whether the audit loss can learn a known gate structure. The digits experiments ask whether the gates can be instantiated as real expert paths. The budget, path, and rule-prior sweeps ask whether soft mixtures and ordered gate paths are more stable than hard gate choice. The later cross-modal, memory, and router/oracle stress tests ask which signals actually explain routing success and when memory claims survive controls.

This organization also clarifies the relationship to larger modular systems. The four-gate controller can later serve as an orchestration layer: the compute gate may call a compute-heavy substrate, the route gate may call a routing fabric, the memory gate may call a memory substrate, and the stop/restart gate may call a trajectory-control substrate. That integration is deliberately left as future work. The present paper first establishes what the controller can and cannot do by itself.

Run note. The toy result does not claim benchmark realism; it shows the audit signal is operational because the system chooses gates, predicts gains, compares alternatives, and reduces regret. The later results make the claim more conservative: the strongest current gains come from stable budgets, ordered paths, rule priors, and dataset-specific signal choices rather than from unconstrained hard gate selection.

9 Toy Prototype Run Summary

What we did. We ran a small CPU-friendly four-gate toy prototype to test the core claim of the paper without adding a larger architecture. The prototype used four gates—Compute, Route, Memory, and Stop/Restart—and generated synthetic examples with four hidden factors: complexity, route need, memory need, and instability. Each hidden factor made one gate more useful, so the experiment had a known best gate for every input.

How we did it. We generated 5,000 training examples and 1,000 test examples. Each example contained noisy observed features, an actual gain value for each of the four gates, and a label for the best gate. A small neural controller received the observed features and produced two outputs: a policy over gates and a predicted gain for each gate. Training used the same objective described in the paper: cross-entropy for choosing the best gate, mean-squared error for aligning predicted gains with actual gains, and regret for penalizing choices worse than the best available gate.

What we got. The controller improved steadily during training. Test gate accuracy increased from 0.520 at epoch 1 to 0.925 at epoch 25. Value-top accuracy increased from 0.665 to 0.928. Gain-prediction RMSE decreased from 0.258 to 0.050. Mean regret decreased from 0.141 to 0.003. The final inspected examples showed entry notes whose selected gates matched the best actual gates, with near-zero regret in the displayed cases.

Epoch	GateAcc	ValueTopAcc	RMSE	Regret
1	0.520	0.665	0.258	0.141
5	0.857	0.848	0.092	0.012
10	0.886	0.882	0.068	0.008
15	0.898	0.907	0.058	0.006
20	0.912	0.922	0.051	0.005
25	0.925	0.928	0.050	0.003

Table 2: Toy prototype output summary. GateAcc measures selected-gate correctness, ValueTopAcc measures whether predicted gains identify the best gate, RMSE measures gain-prediction alignment, and Regret measures missed value relative to the best gate.

What it means. The toy result supports the narrow mechanism claim: a gate-logged controller can learn not only which internal action to choose, but also how to predict the gain of each option. The drop in regret means the controller increasingly avoids costly gate choices. The drop in RMSE means its expected gains become closer to observed gains. Together, these results show that entry-note predictions and exit-note outcomes can become measurable training signals.

What lesson we learned. The main lesson is that the four-gate controller should be evaluated first as a standalone decision-calibration system. The toy prototype does not prove performance on real benchmarks, but it shows that the proposed training signal is operational: gate choice, predicted gain, observed gain, and counterfactual regret can all be logged and optimized. The next step is therefore not to add complexity immediately, but to run baselines and ablations that remove notes, alignment loss, regret loss, and individual gates.

10 Digits Real-Data Sanity Test Summary

What we did. We ran real-data sanity tests on sklearn digits to check whether gate-logged control works when gate gains come from real classifiers. The gates were real expert paths: Compute used an MLP, Route used PCA with logistic regression, Memory used KNN, and Stop/Restart used a shallow decision tree. Expert test accuracies were Compute = 0.939, Route = 0.942, Memory = 0.961, and Stop/Restart = 0.828, with Memory as the best single expert.

How we did it. The split used 1,077 expert-training examples, 360 gate-training examples, and 360 test examples. The single-seed test used true-label probability minus cost as gain. The multi-

factor test used three utility settings—accuracy-first, balanced, and efficiency-first—where utility combined correctness, true-label probability, prediction margin, entropy, and cost. Each setting was trained over five seeds: 1, 7, 21, 42, and 88.

What we got. The single-seed controller reached $\text{TaskAcc} = 0.906$, $\text{GateAcc} = 0.686$, $\text{ValueTopAcc} = 0.336$, $\text{RMSE} = 0.277$, and $\text{Regret} = 0.097$. In the multi-seed test, the balanced setting had the best mean task accuracy: $\text{TaskAcc} = 0.8533 \pm 0.0048$, $\text{GateAcc} = 0.7717 \pm 0.0110$, $\text{Regret} = 0.0921 \pm 0.0035$, $\text{ValueTopAcc} = 0.4283 \pm 0.0535$, $\text{RMSE} = 0.2326 \pm 0.0024$, $\text{AvgCost} = 0.0123 \pm 0.0003$, and $\text{OracleTask} = 0.9917$.

Setting	Task	Gate	Regret	RMSE	Cost
Balanced	.853±.005	.772±.011	.092±.004	.233±.002	.012±.000
Accuracy	.853±.003	.771±.010	.118±.003	.268±.003	.012±.000
Efficiency	.828±.001	.819±.000	.057±.001	.166±.007	.010±.000

Table 3: Multi-factor digits test over five seeds. Balanced gives the best mean task accuracy; efficiency-first gives the lowest regret and cost.

What it means. The multi-seed result is more honest than a single run: the controller learns a stable non-random gate policy, but it still does not beat the best single expert. Balanced performs best for task accuracy, while efficiency-first reduces regret and cost because it chooses Stop/Restart most often. The top balanced seeds selected Stop/Restart around 86–89% of the time and Route around 11–14%, showing that the learned policy is cost-sensitive and not yet fully diverse.

What lesson we learned. The real-data tests support the controller as an auditable routing mechanism, not yet as a superior accuracy system. The next improvement should target stronger controller features, more gate-training data, better calibration of value predictions, and explicit anti-collapse regularization so the policy does not overuse a cheap gate when higher-accuracy experts are available.

11 Ten-Pass Episodic Gate Test Summary

What we did. We tested whether a gate controller improves when it can act over a 10-step episode instead of making one static decision. The comparison used the same sklearn digits experts and the balanced multi-factor utility. The static controller chose one gate once, while the episodic controller used a recurrent hidden state, previous gate, previous reward, and step index to choose gates over 10 passes.

How we did it. Each episode applied a small penalty for repeating the same gate, with an additional penalty when the previous reward was weak. This created a simple trajectory-control setting: the controller could continue with the same gate, switch routes, or fall back to Stop/Restart depending on previous outcomes. The logs recorded entry gates, confidence, predicted gain, actual gain, best gate, regret, and task correctness at every step.

What we got. The static one-pass controller reached $\text{TaskAcc} = 0.8556$, $\text{GateAcc} = 0.7806$, $\text{Regret} = 0.0901$, $\text{ValueTopAcc} = 0.3889$, and $\text{RMSE} = 0.2225$, choosing Stop/Restart 89.2% of the time and Route 10.8%. The episodic 10-pass controller reached mean $\text{TaskAcc} = 0.8847$, mean $\text{Regret} = 0.0982$, mean $\text{Reward} = 0.7812$, and mean $\text{Switches} = 0.23$ per episode, with gate usage shifting to Stop/Restart 67.2%, Route 23.1%, and Memory 9.7%.

Controller	TaskAcc	Regret	Reward	Switches	Main Gate Use
Static 1-pass	.856	.090	–	–	Stop/Restart 89.2%
Episodic 10-pass	.885	.098	.781	.23	Stop 67.2%, Route 23.1%, Memory 9.7%

Table 4: Static versus episodic gate control on digits. Episodic control improves task accuracy and gate diversity, but slightly increases regret.

What it means. The 10-pass controller helped final task accuracy by $+0.0292$ and reduced collapse into a single gate, but it did not cleanly improve regret. The best inspected episode switched almost every step and achieved perfect task correctness with low regret; the worst episode repeatedly chose Stop/Restart when Route was best, causing high regret and zero task accuracy. This shows that episodic memory can help, but weak recovery behavior can also amplify bad early choices.

What lesson we learned. The episodic test supports adding trajectory control as a serious fourth gate: history changes the routing policy and can improve task performance. However, the controller still needs stronger switch triggers, better low-reward recovery, and anti-collapse regularization so repeated Stop/Restart choices do not dominate failure cases.

12 Hybrid Rule-and-Learned Gate Test Summary

What we did. We added a multi-rule and hybrid controller test on sklearn digits to compare hard rules, pure learned policy, rule-regularized learning, and hybrid rule-plus-learned selection. The same four experts were used: MLP for Compute, PCA–logistic regression for Route, KNN for Memory, and decision tree for Stop/Restart.

How we did it. The controller used cheap image features such as intensity, active-pixel ratio, center of mass, edge energy, symmetry, and spread. Three rule modes were tested: balanced, conservative, and accuracy. The hybrid policy mixed learned utility predictions with rule bonuses and used rule overrides when the learned policy was uncertain. Results were averaged over five seeds.

What we got. The best configuration was *hybrid-accuracy*, with TaskAcc = 0.9622 ± 0.0014 , GateAcc = 0.0950 ± 0.0044 , Regret = 0.1012 ± 0.0023 , AvgCost = 0.0480 ± 0.0006 , ValueTopAcc = 0.3150 ± 0.0221 , and RMSE = 0.2481 ± 0.0095 . This slightly exceeded the best single expert Memory accuracy of 0.9611. The next best was rule-only-accuracy at 0.9611, followed by hybrid-balanced at 0.9117.

Config	TaskAcc	GateAcc	Regret	AvgCost
Hybrid-accuracy	.962 $\pm$.001	.095 $\pm$.004	.101 $\pm$.002	.048 $\pm$.001
Rule-only accuracy	.961 $\pm$.000	.089 $\pm$.000	.104 $\pm$.000	.049 $\pm$.000
Hybrid-balanced	.912 $\pm$.001	.238 $\pm$.001	.112 $\pm$.001	.038 $\pm$.000

Table 5: Top hybrid/rule configurations on sklearn digits over five seeds. Hybrid-accuracy gives the best task accuracy, while its low gate accuracy shows that task performance and utility-target matching can diverge.

What it means. This is the strongest task-accuracy result so far: hybrid-accuracy slightly beats the best single expert. However, the low gate accuracy and gate distribution show why the claim

must stay careful. The best seeds chose Memory about 76–78% of the time, with smaller use of Route and Compute, so the hybrid rule is acting partly like a strong expert-selection bias rather than a fully balanced four-gate policy.

What lesson we learned. Rules can help stabilize and improve the learned controller, especially when the learned policy is uncertain. The next version should keep the hybrid mechanism, but add diversity constraints, better gate-target design, and explicit reporting of both task accuracy and gate accuracy so improvements do not hide gate collapse.

13 Note-Order Read/Write Test Summary

What we did. We tested whether the order and number of note operations matter by comparing six schedules: one-pass read-then-write (RW_1x), one-pass write-then-read (WR_1x), and four two-pass schedules (RW_RW, WR_WR, RW_WR, WR_RW). The goal was to see whether extra note cycles improve gate choice or merely add noise.

How we did it. Each controller used cheap digit-image features and a learned note state. A reader produced context from the current input and note, while a writer updated the note. The policy then selected one of the four gates. We trained each schedule over three seeds using the same task, alignment, and regret losses.

What we got. All schedules produced the same task accuracy, gate accuracy, and regret: TaskAcc = 0.8278, GateAcc = 0.7611, and Regret = 0.1355. The best-ranked schedule by task accuracy was RW_1x, but it did not truly outperform the others. Its top seeds all selected Stop/Restart 100% of the time, showing complete gate collapse. Two-pass schedules also failed to improve over one-pass schedules: one-pass mean TaskAcc was 0.8278 ± 0.0000 , two-pass mean TaskAcc was 0.8278 ± 0.0000 , and both had mean Regret = 0.1355 ± 0.0000 .

Schedule	TaskAcc	GateAcc	Regret	RMSE	Note Shift
RW_1x	.828	.761	.136	.272	2.059
WR_1x	.828	.761	.136	.276	1.536
RW_RW	.828	.761	.136	.261	1.310

Table 6: Top note-order schedules. Extra read/write cycles did not improve task accuracy or regret and produced Stop/Restart collapse in the best-ranked one-pass schedule.

What it means. This is an important negative result. Merely adding note read/write cycles does not guarantee better internal control. The note mechanism changed internal note states, but the policy still collapsed to the cheap Stop/Restart gate, so the notes were not functionally improving gate diversity or task performance.

What lesson we learned. Entry and exit notes need explicit anti-collapse pressure, stronger supervision, or diversity constraints. Otherwise, a controller may learn to write and read notes while still making the same gate decision every time. This supports the paper’s main caution: note logging is only useful when the logged prediction is tied to measurable decision improvement.

14 Gate-Budget Sweep Summary

What we did. We tested fixed percentage budgets across the four gates on sklearn digits to answer a simple allocation question: does giving more weight to Memory help, and which split is most stable across seeds? The evaluated splits included balanced allocation, several Memory-heavy allocations, Route-heavy, Compute-heavy, and Stop/Restart-heavy settings.

How we did it. For each seed, the four expert gates produced probability distributions over digit labels. A budget split formed a weighted mixture of those expert probabilities, and the final digit prediction was the argmax of the mixture. We evaluated five seeds and reported task accuracy, regret, average cost, confidence, entropy, best single expert accuracy, and oracle best-gate accuracy.

What we got. The best split was Memory-40: Compute 20%, Route 20%, Memory 40%, and Stop/Restart 20%. It reached TaskAcc = 0.9761 ± 0.0042 , Regret = 0.1032 ± 0.0010 , AvgCost = 0.0440, AvgConfidence = 0.8876 ± 0.0097 , and AvgEntropy = 0.1526 ± 0.0191 . This beat the best single expert accuracy of 0.9611 and approached the oracle best-gate accuracy of about 0.9922. The next two splits were Memory-55 at 0.9722 ± 0.0035 and balanced-25 at 0.9717 ± 0.0048 .

Split	Weights C/R/M/S	TaskAcc	Regret	Cost
Memory-40	20/20/40/20	$.976 \pm .004$	$.103 \pm .001$	.044
Memory-55	15/15/55/15	$.972 \pm .004$	$.103 \pm .001$	.046
Balanced-25	25/25/25/25	$.972 \pm .005$	$.225 \pm .035$	.043

Table 7: Top gate-budget splits over five seeds. A moderate Memory-heavy mixture performs best, while too much or too little Memory is less effective.

What it means. This result is stronger than selecting one expert or one hard gate. A moderate Memory-heavy mixture outperformed the best single expert, suggesting that the four gates can be useful as a budgeted ensemble even when learned gate selection is unstable. Memory helps most when it is emphasized but not allowed to dominate completely.

What lesson we learned. The controller should not only choose a discrete gate; future versions should also learn soft budget allocation across gates. The best result came from sharing budget across all four gates while giving Memory the largest share, so the next design should treat gate control as both a routing problem and a resource-allocation problem.

15 Connected-Room Two-Gate Path Sweep Summary

What we did. We tested the connected-room idea by evaluating all 16 ordered two-gate paths $A \rightarrow B$, where A and B are Compute, Route, Memory, or Stop/Restart. The question was whether entering one gate first and then moving to a second gate improves final prediction compared with a single expert.

How we did it. Each path mixed the probability output of room A with room B . The second room received more weight when the first room had favorable confidence, entropy, margin, or when the path switched rooms. We ran the sweep over three seeds and reported task accuracy, regret, true-label probability, confidence, transition weight β , cost, best single-expert accuracy, and oracle path accuracy.

What we got. The best path was Stop/Restart→Memory with TaskAcc = 0.9704 ± 0.0013 , Regret = 0.0878 ± 0.0004 , AvgTrueProb = 0.8906 ± 0.0014 , AvgConfidence = 0.9000 ± 0.0012 , AvgBeta = 0.7544 , and AvgCost = 0.0650 . This beat the best single expert accuracy of 0.9611 and remained below the oracle path accuracy of 0.9926 ± 0.0013 . The next best paths were Compute→Memory at 0.9657 and Memory→Compute at 0.9648 .

Path	TaskAcc	Regret	TrueProb	Cost
Stop/Restart→Memory	$.970 \pm .001$	$.088 \pm .000$	$.891 \pm .001$	.065
Compute→Memory	$.966 \pm .003$	$.092 \pm .024$	$.886 \pm .025$	.135
Memory→Compute	$.965 \pm .011$	$.124 \pm .041$	$.855 \pm .042$	.135

Table 8: Top connected-room two-gate paths over three seeds. Paths ending in Memory perform best, suggesting that a second retrieval-like room often improves the final prediction.

What it means. The connected-room result supports the idea that gates can be composed rather than chosen only once. Stop/Restart→Memory is especially interesting: the cheap conservative first room followed by retrieval-like Memory outperformed the best single expert, suggesting that a staged path can improve accuracy while keeping cost moderate.

What lesson we learned. The controller should eventually learn ordered gate paths, not just single gates or static budgets. The best paths all involved Memory, which agrees with the budget sweep: Memory is useful, but it works best when paired with another gate or given a moderate share of the total budget.

16 Combined Large-Sweep Summary

What we did. We combined the strongest previous ideas on sklearn digits: Memory-heavy weighted mixture, Stop/Restart→Memory path, and a hybrid of both. The goal was to test stability across seeds and training-data scales.

How we did it. We evaluated five seeds and three expert-training scales: 0.40, 0.70, and 1.00. For each scale and seed, experts were retrained, then three strategies were evaluated: Memory-mix, Stop/Restart→Memory path, and Hybrid. Metrics included task accuracy, regret, confidence, entropy, cost, best single-expert accuracy, and oracle best-expert accuracy.

What we got. The best overall configuration was Memory-mix at scale 1.00, with TaskAcc = 0.9761 ± 0.0042 , Regret = 0.0976 ± 0.0091 , AvgConfidence = 0.8876 ± 0.0097 , AvgEntropy = 0.1526 ± 0.0191 , AvgCost = 0.0440 , BestSingleAcc = 0.9611 , and OracleBestAcc = 0.9922 ± 0.0011 . Hybrid at scale 1.00 reached TaskAcc = 0.9700 ± 0.0037 and Regret = 0.0896 ± 0.0030 , while Stop/Restart→Memory reached TaskAcc = 0.9700 ± 0.0021 and Regret = 0.0887 ± 0.0028 .

Config	TaskAcc	Regret	Conf.	Cost
Memory-mix @ 1.00	$.976 \pm .004$	$.098 \pm .009$	$.888 \pm .010$	.044
Hybrid @ 1.00	$.970 \pm .004$	$.090 \pm .003$	$.897 \pm .003$	.055
Stop→Memory @ 1.00	$.970 \pm .002$	$.089 \pm .003$	$.900 \pm .002$	.065

Table 9: Combined large sweep over five seeds and three data scales. Full-scale Memory-mix gives the best task accuracy, while path and hybrid variants reduce regret.

Scale behavior. The hybrid strategy improved as more training data were available: at scale 0.40 it reached $\text{TaskAcc} = 0.9422 \pm 0.0044$ and $\text{Regret} = 0.1298 \pm 0.0042$; at scale 0.70 it reached $\text{TaskAcc} = 0.9633 \pm 0.0059$ and $\text{Regret} = 0.1051 \pm 0.0046$; at scale 1.00 it reached $\text{TaskAcc} = 0.9700 \pm 0.0037$ and $\text{Regret} = 0.0896 \pm 0.0030$.

What it means. The strongest result remains the simple Memory-heavy budgeted mixture, not the most complex controller. However, the connected path and hybrid strategies have lower regret, indicating that path structure may improve calibration even when it does not maximize task accuracy.

What lesson we learned. The first paper should report the honest hierarchy: soft budget allocation currently gives the best accuracy, connected paths give competitive accuracy with lower regret, and learned controllers need stronger stability mechanisms before they can reliably beat simple mixtures. This supports treating gate control as both a budget-allocation and path-selection problem.

17 Multi-Dataset Routing Sweep Summary

What we did. We tested whether the strongest digits strategies transfer beyond sklearn digits. The sweep used three datasets: CIFAR-10 for images, IMDB for sentiment, and AG News for topic classification. For each dataset, we compared Memory-mix, Stop/Restart→Memory path, and Hybrid across multiple seeds and training sizes.

How we did it. CIFAR-10 used lightweight image experts based on pooled grayscale features, color statistics, logistic regression, KNN, Gaussian Naive Bayes, and decision trees. IMDB and AG News used text experts based on word TF-IDF, word bigram TF-IDF, character n-grams, logistic/SGD classifiers, and Complement Naive Bayes. For each dataset, we measured task accuracy, regret, confidence, entropy, cost, best single-expert accuracy, and oracle best-expert accuracy.

What we got. Memory-mix was the best configuration on all three datasets at the largest training size. On CIFAR-10, Memory-mix at $n = 5000$ reached $\text{TaskAcc} = 0.2924 \pm 0.0090$, above the best single expert at 0.2793 ± 0.0028 . On IMDB, Memory-mix at $n = 10000$ reached $\text{TaskAcc} = 0.8562 \pm 0.0036$, below the best single expert at 0.8746 ± 0.0051 but above the Hybrid at 0.8472 ± 0.0042 . On AG News, Memory-mix at $n = 15000$ reached $\text{TaskAcc} = 0.9003 \pm 0.0003$, slightly above the best single expert at 0.8959 ± 0.0013 and above Hybrid at 0.8974 ± 0.0011 .

Dataset	Best Config	TaskAcc	Regret	Best Single
CIFAR-10	Memory-mix @ 5000	.292 ± .009	.273 ± .003	.279 ± .003
IMDB	Memory-mix @ 10000	.856 ± .004	.160 ± .001	.875 ± .005
AG News	Memory-mix @ 15000	.900 ± .000	.198 ± .001	.896 ± .001

Table 10: Multi-dataset routing sweep. Memory-mix is the best tested strategy on all three datasets, though it does not beat the best single expert on IMDB.

Scale behavior. Hybrid improved with larger training sizes on all datasets: CIFAR-10 rose from 0.2439 to 0.2613, IMDB from 0.8225 to 0.8472, and AG News from 0.8755 to 0.8974. This suggests that the hybrid strategy benefits from scale, even when Memory-mix remains the top strategy.

What it means. The four-gate idea is not limited to digits. Across image and text domains,

the Memory-heavy mixture remains a strong and simple baseline. However, results are dataset-dependent: it beats the best single expert on CIFAR-10 and AG News, but not on IMDB.

What lesson we learned. The paper should treat Memory-mix as a strong baseline, but not as a universal cross-domain solution; no-memory, disagreement-aware, and rule-prior mixtures can be better when the modality and expert-agreement structure favor them. The honest next step is to improve learned or hybrid gate control until it consistently beats the simple Memory-heavy mixture and the best single expert across domains.

18 Weakness-Hunt Text Routing Sweep Summary

What we did. We ran a targeted text-domain weakness hunt on IMDB and AG News to check whether the apparent Memory-mix advantage was robust to alternative text-routing configurations. The sweep compared memory-only routing, no-memory mixtures, and related text expert combinations at the largest tested training sizes.

What we got. On IMDB, the best weakness-hunt configuration was `memory_only` at $n = 10000$, with $\text{TaskAcc} = 0.8640 \pm 0.0022$. This improved over the earlier Memory-mix result of 0.8562 ± 0.0036 but still remained below the best single text expert at 0.8732, suggesting that the limiting factor is mostly data scarcity or representation strength rather than routing alone. On AG News, the best weakness-hunt configuration was `no_memory_mix` at $n = 15000$, with $\text{TaskAcc} = 0.8966 \pm 0.0011$. This beat the best single expert at 0.8910, but did so without Memory, suggesting that the learned routing/controller side remains underfit or misallocated for this dataset.

Dataset	Best Config	Train n	TaskAcc	Best Single	Diagnosis
IMDB	<code>memory_only</code>	10000	$.864 \pm .002$	.873	data scarcity
AG News	<code>no_memory_mix</code>	15000	$.897 \pm .001$	.891	controller underfit

Table 11: Weakness-hunt text routing sweep. The best text configuration is dataset-dependent: IMDB favors memory-only routing but remains below the best single expert, while AG News performs best without Memory.

What it means. Memory is not universally the best gate. It is a strong baseline on several tasks, but the weakness hunt shows that the right routing policy depends on dataset structure, training size, and expert quality. Future experiments should therefore report dataset-specific failure diagnoses rather than treating a single gate mixture as the global solution.

19 Cross-Modal Policy-Chooser Benchmark Summary

What we did. We ran a CPU-only cross-modal benchmark to test whether the gate idea transfers beyond a single modality and whether a new policy-of-policies controller can choose among candidate routing strategies. The benchmark used Breast Cancer as a tabular task, FashionMNIST as an image task, and 20 Newsgroups as a text task. For each dataset, we tested three training densities and three seeds. The candidate strategies were `no_memory_mix`, `memory_mix`, `disagreement_mix`, `Stop/Restart→Memory`, `learned_selector`, and `policy_chooser`.

How we did it. Each dataset used four lightweight experts mapped onto the four-gate vocabulary.

Breast Cancer used logistic regression, KNN, decision tree, and Gaussian Naive Bayes; Fashion-MNIST used logistic regression, KNN, decision tree, and Gaussian Naive Bayes over flattened pixels; and 20 Newsgroups used word TF-IDF, word bigram TF-IDF, count-vector Naive Bayes, and character n-gram TF-IDF experts. The policy chooser was trained on meta-features describing expert confidence, margin, entropy, pairwise agreement, modality features, and candidate-policy confidence and entropy.

What we got. The best strategy was modality-dependent. On Breast Cancer, `no_memory_mix` at full density reached $\text{TaskAcc} = 0.9678 \pm 0.0041$, below the best single expert at 0.9737 but with low regret $= 0.0146 \pm 0.0083$. On FashionMNIST, `disagreement_mix` at train fraction 0.12 reached $\text{TaskAcc} = 0.8134 \pm 0.0043$, above the best single expert at 0.8002 ± 0.0050 , with an oracle ceiling of 0.9362 ± 0.0029 . On 20 Newsgroups, `disagreement_mix` at train fraction 0.20 reached $\text{TaskAcc} = 0.5908 \pm 0.0104$, above the best single expert at 0.5671 ± 0.0140 , with an oracle ceiling of 0.6758 ± 0.0036 .

Dataset	Best Config	Density	TaskAcc	Best Single	Oracle
Breast Cancer	<code>no_memory_mix</code>	1.00	$.968 \pm .004$	.974	$.991 \pm .007$
FashionMNIST	<code>disagreement_mix</code>	0.12	$.813 \pm .004$	$.800 \pm .005$	$.936 \pm .003$
20 Newsgroups	<code>disagreement_mix</code>	0.20	$.591 \pm .010$	$.567 \pm .014$	$.676 \pm .004$

Table 12: Cross-modal policy-chooser benchmark over tabular, image, and text datasets. The strongest policy differs by modality: tabular data favors a no-memory mixture, while image and text tasks favor disagreement-aware mixing.

Policy-chooser behavior. The learned `policy_chooser` did not become the best overall strategy. It was competitive on Breast Cancer, reaching $\text{TaskAcc} = 0.9561 \pm 0.0189$ at full density, but underperformed simple disagreement or no-memory mixtures on FashionMNIST and 20 Newsgroups. This indicates that a policy-of-policies layer is not automatically better than a hand-designed mixture unless the meta-training signal is strong enough and the candidate policies are separable.

Density behavior. More data helped most clearly for FashionMNIST and 20 Newsgroups. FashionMNIST `disagreement_mix` improved from 0.7799 at density 0.03 to 0.8134 at density 0.12, while 20 Newsgroups `disagreement_mix` improved from 0.4481 at density 0.05 to 0.5908 at density 0.20. Breast Cancer was already high at low density, so density gains were smaller and noisier.

Weakness diagnostics. The dominant weakness changed by modality. Breast Cancer had high expert agreement (0.9370) and a low Memory oracle rate (0.1589), so no-memory mixing was strongest and Memory was mostly redundant. FashionMNIST had lower expert agreement (0.6327), a large oracle gap, and clear density gains, so the main weakness was data density and undertrained visual experts. 20 Newsgroups had low expert agreement (0.5146) and a high Memory oracle rate (0.5929), so Memory remained important, but disagreement-aware mixing beat plain Memory-mix.

What it means. The cross-modal benchmark strengthens the paper’s conservative claim. Gate-logged control should not assume that Memory-heavy routing is always best, nor that an extra learned chooser will automatically dominate. Across modalities, the best current rule is diagnostic: use no-memory mixtures when experts already agree and Memory is rarely oracle-best, use disagreement-aware mixtures when experts disagree, and reserve learned policy choice for settings with enough meta-data to close the oracle gap.

20 Rule-First, Chooser-Second Benchmark Summary

What we did. We tested a GLAMARC-style rule-first, chooser-second routing design on the same three cross-modal datasets: Breast Cancer, FashionMNIST, and 20 Newsgroups. The benchmark kept the same candidate strategies as the previous cross-modal sweep but added a fixed rule_mix baseline and two learned choosers: plain_policy_chooser and rule_policy_chooser. The rule prior was dataset-specific: tabular data favored no-memory mixing, image data favored disagreement-aware mixing, and text data favored a mixture of disagreement and Memory.

How we did it. For each dataset, we trained four lightweight experts and built candidate strategy distributions. The rule_mix baseline averaged candidate policies using the dataset prior. The plain chooser used candidate and meta-features without a rule bias, while the rule-aware chooser received the same features plus the prior. We evaluated three training densities and three seeds, using task accuracy, regret, confidence, entropy, cost, best single-expert accuracy, and oracle best-expert accuracy.

What we got. On Breast Cancer, no_memory_mix at full density remained best, with TaskAcc = 0.9678 ± 0.0041 , Regret = 0.0146 ± 0.0083 , and BestSingleAcc = 0.9737. Both plain_policy_chooser and rule_policy_chooser reached 0.9591 ± 0.0165 , so the rule prior did not improve over the plain chooser. On FashionMNIST, rule_mix at density 0.12 became the best strategy, with TaskAcc = 0.8189 ± 0.0048 , Regret = 0.0731 ± 0.0032 , and BestSingleAcc = 0.8002 ± 0.0050 . On 20 Newsgroups, rule_mix at density 0.20 also became best, with TaskAcc = 0.5921 ± 0.0110 , Regret = 0.0950 ± 0.0107 , and BestSingleAcc = 0.5671 ± 0.0140 .

Dataset	Best Config	Density	TaskAcc	Regret	Best Single
Breast Cancer	no_memory_mix	1.00	$.968 \pm .004$	$.015 \pm .008$	.974
FashionMNIST	rule_mix	0.12	$.819 \pm .005$	$.073 \pm .003$	$.800 \pm .005$
20 Newsgroups	rule_mix	0.20	$.592 \pm .011$	$.095 \pm .011$	$.567 \pm .014$

Table 13: Rule-first, chooser-second benchmark. A fixed rule prior improves the image and text settings, while the tabular setting still favors the simpler no-memory mixture.

Rule-prior effect. The rule prior helped most when it was used as a fixed mixture rather than as an extra feature for a learned chooser. On FashionMNIST, rule_mix improved over disagreement_mix (0.8189 versus 0.8134) and reduced regret (0.0731 versus 0.0791). On 20 Newsgroups, rule_mix slightly improved over disagreement_mix (0.5921 versus 0.5908) and memory_mix (0.5882). On Breast Cancer, however, the handcrafted prior diluted the strongest no-memory policy, so rule_mix reached only 0.9503 at full density.

Chooser behavior. The rule_policy_chooser matched the plain_policy_chooser exactly in these runs. On Breast Cancer both reached 0.9591 ± 0.0165 at full density; on FashionMNIST both fell to 0.7578 at density 0.12; and on 20 Newsgroups both reached 0.5739 at density 0.20. This suggests that simply appending a static rule prior to the chooser features is too weak to change the learned decision boundary. The rule helps when it directly shapes the mixture, not when it is only a passive input to a small classifier.

Density behavior. The rule-first benchmark again showed strong density sensitivity outside the tabular task. FashionMNIST rule_mix improved from 0.7861 at density 0.03 to 0.8189 at density 0.12. 20 Newsgroups rule_mix improved from 0.4477 at density 0.05 to 0.5921 at density 0.20. Breast Cancer changed less, rising from 0.9386 to 0.9503 for rule_mix while no_memory_mix stayed

the best overall.

What it means. The rule-first result supports a modest design update: use human-readable routing priors as direct mixture constraints before asking a learned chooser to arbitrate. The fixed rule prior gives small but consistent gains on image and text tasks, while the learned chooser still underfits or ignores the rule signal. The next controller should therefore treat rules as executable policy structure, not merely as metadata appended to a classifier.

21 Note-Format Routing Benchmark Summary

What we did. We tested whether the format of the audit note changes routing performance in a GLAMARC-style chooser. The benchmark compared four text note formats—JSON, table, Q&A, and compact text—against a structured chooser and the fixed rule_mix baseline. Each note was generated per sample, so every field was scalar rather than a batch-level array. The same three datasets were used: Breast Cancer, FashionMNIST, and 20 Newsgroups.

How we did it. Each note encoded the dataset, modality, confidence and entropy signals, pairwise expert agreement, Memory-specific signals, rule priors, candidate scores, candidate confidence, candidate entropy, and a best-hint field. Text readers used character n-gram TF-IDF plus logistic regression to predict which candidate strategy should be selected. The structured chooser received the same information as numeric features. We evaluated three densities and three seeds, then compared task accuracy, regret, selection accuracy, confidence, entropy, cost, best single-expert accuracy, and the reported oracle diagnostic.

What we got. The best note format depended on modality. On Breast Cancer, JSON notes at full density were best, reaching TaskAcc = 0.9678 ± 0.0041 and Regret = 0.0146 ± 0.0083 , matching the best no-memory result from the earlier sweep while remaining below the best single expert at 0.9737. On FashionMNIST, rule_mix remained best at density 0.12, with TaskAcc = 0.8193 ± 0.0045 and Regret = 0.0727 ± 0.0031 ; the best note reader was JSON at 0.8150 ± 0.0009 . On 20 Newsgroups, table notes were best at density 0.20, with TaskAcc = 0.5649 ± 0.0089 and Regret = 0.0890 ± 0.0033 , narrowly above rule_mix at 0.5648 ± 0.0048 .

Dataset	Best Method	Density	TaskAcc	Regret	Best Single
Breast Cancer	json_note_reader	1.00	.968 $\pm$.004	.015 $\pm$.008	.974
FashionMNIST	rule_mix	0.12	.819 $\pm$.005	.073 $\pm$.003	.800 $\pm$.005
20 Newsgroups	table_note_reader	0.20	.565 $\pm$.009	.089 $\pm$.003	.617 $\pm$.010

Table 14: Note-format routing benchmark. JSON notes work best on the tabular task, the fixed rule mixture remains best on FashionMNIST, and table notes are narrowly best on 20 Newsgroups.

Format behavior. On Breast Cancer, all text note readers beat the structured chooser on average: JSON reached mean TaskAcc = 0.9561, compact text = 0.9542, table = 0.9522, Q&A = 0.9513, and the structured chooser = 0.9474. On FashionMNIST, rule_mix and JSON were close, with average accuracies 0.8025 and 0.8014, while the structured chooser was much lower at 0.7629. On 20 Newsgroups, format mattered only mildly: rule_mix averaged 0.4966, structured features 0.4929, table notes 0.4903, Q&A 0.4883, compact notes 0.4875, and JSON 0.4816.

Density behavior. The note-format sweep again showed that density matters. JSON notes on Breast Cancer rose from 0.9532 at density 0.25 to 0.9678 at full density. FashionMNIST rule_mix

rose from 0.7870 at density 0.03 to 0.8193 at density 0.12, while JSON notes rose from 0.7881 to 0.8150. On 20 Newsgroups, table notes rose from 0.4224 at density 0.05 to 0.5649 at density 0.20.

What it means. Note layout can matter, but it is not the only bottleneck. JSON and compact notes help when structured scalar fields are easy to parse, table notes help slightly on the text task, and the fixed rule mixture remains strong when the hand-designed prior already matches the modality. The safest conclusion is that audit notes should be machine-readable and scalarized, but format alone cannot replace better candidate strategies or stronger routing supervision.

22 Note-Memory Routing Benchmark Summary

What we did. We tested whether a JSON-note router improves when it can also read a running memory of prior mistakes. The memory state tracked recent correctness, regret, confidence, entropy, chosen gate, best gate, and gate-specific indicators. The benchmark compared `json_note_only` against `json_note_plus_memory` on Breast Cancer, FashionMNIST, and 20 Newsgroups across three densities and three seeds.

How we did it. Each input produced a current JSON audit note containing dataset, modality, expert predictions, top-1 probabilities, margins, entropies, pairwise agreement, gate names, gate predictions, and cost-adjusted proxy scores. A note-only router was trained from the JSON text to predict the best cost-adjusted gate. A second router received the same note representation plus a compact exponentially smoothed memory vector updated from prior routing mistakes. Evaluation used a fixed random test order so the memory model had a sequential stream of previous outcomes.

What we got. On Breast Cancer, `json_note_only` at density 0.25 was the best task-accuracy setting, with $\text{TaskAcc} = 0.9591 \pm 0.0083$, $\text{GateAcc} = 0.0175 \pm 0.0124$, and $\text{Regret} = 0.0972 \pm 0.0224$. The memory version matched its task accuracy at density 0.25 and improved the full-density setting from 0.9357 to 0.9474, with lower full-density regret (0.0531 versus 0.0628). On FashionMNIST, `json_note_only` was clearly stronger: at density 0.12 it reached $\text{TaskAcc} = 0.7739 \pm 0.0065$, while `json_note_plus_memory` reached 0.7612 ± 0.0226 . On 20 Newsgroups, memory gave a small top-density gain: `json_note_plus_memory` at density 0.20 reached $\text{TaskAcc} = 0.5543 \pm 0.0161$, slightly above `json_note_only` at 0.5511 ± 0.0156 .

Dataset	Best Method	Density	TaskAcc	GateAcc	Regret
Breast Cancer	<code>json_note_only</code>	0.25	$.959 \pm .008$	$.018 \pm .012$	$.097 \pm .022$
FashionMNIST	<code>json_note_only</code>	0.12	$.774 \pm .007$	$.697 \pm .004$	$.162 \pm .008$
20 Newsgroups	<code>json_note_plus_memory</code>	0.20	$.554 \pm .016$	$.026 \pm .036$	$.149 \pm .004$

Table 15: Note-memory routing benchmark. Memory is mildly helpful on Breast Cancer and 20 Newsgroups, but hurts FashionMNIST; the effect is weaker than the effect of choosing good candidate policies.

Memory behavior. Averaged across densities, Breast Cancer showed a small memory benefit: mean TaskAcc rose from 0.9454 to 0.9474, mean GateAcc rose from 0.2173 to 0.2242, and mean Regret fell from 0.0859 to 0.0836. FashionMNIST showed the opposite pattern: mean TaskAcc fell from 0.7485 to 0.7433, mean GateAcc fell from 0.6721 to 0.6448, and mean Regret rose from 0.1799 to 0.1854. On 20 Newsgroups, memory was nearly neutral but slightly positive on average: mean TaskAcc rose from 0.4792 to 0.4801, mean GateAcc rose from 0.0827 to 0.0950, and mean Regret

fell from 0.1401 to 0.1382.

Density behavior. FashionMNIST improved cleanly with density for both methods, but note-only remained ahead at every density. 20 Newsgroups also improved strongly with density, rising from about 0.41 at density 0.05 to about 0.55 at density 0.20. Breast Cancer was less monotonic: the low-density note-only setting had the best task accuracy, while full-density memory produced better gate accuracy and lower regret.

What it means. A running memory of prior mistakes is not automatically useful. It can slightly improve gate selection and regret on some datasets, but it can also inject harmful history bias when the current note already contains enough signal. The next memory design should therefore learn when to trust history, perhaps by gating memory access or resetting memory when the input distribution shifts.

23 Reason-Hunt Stress-Test Summary

What we did. We ran a GLAMARC reason-hunt stress test to separate five possible explanations for routing behavior: note format, memory of prior mistakes, shuffled or noisy memory controls, data density, and seed stability. The test reused Breast Cancer, FashionMNIST, and 20 Newsgroups. It compared four note formats—JSON, table, Q&A, and compact text—and five memory modes: no memory, running memory, shuffled memory, reset-every-four memory, and noise memory.

How we did it. For each sample, the note encoded modality, pairwise agreement, mean top-1 confidence, mean margin, mean entropy, ranked gate candidates, and a best-hint gate. The note reader predicted the best cost-adjusted gate. The stress-test then selected the best note format on the gate split and evaluated memory variants built from the corresponding note stream. Running memory used the real sequential mistake state; shuffled and noise controls tested whether improvements came from meaningful history or from extra tokens/features.

What we got. On Breast Cancer, JSON at density 0.50 was the best note format, with TaskAcc = 0.9444 ± 0.0149 , GateAcc = 0.9064 ± 0.0083 , and Regret = 0.0435 ± 0.0135 . Running memory at density 0.50 slightly improved task accuracy to 0.9474 ± 0.0124 , but shuffled and noise memory reached the same task accuracy, so the real memory signal was not clearly causal. On FashionMNIST, compact notes at density 0.12 were best, with TaskAcc = 0.7834 ± 0.0037 , GateAcc = 0.6881 ± 0.0105 , and Regret = 0.1562 ± 0.0049 ; the best memory mode was no memory, so history hurt rather than helped. On 20 Newsgroups, JSON at density 0.20 was best among note formats, with TaskAcc = 0.5232 ± 0.0089 , GateAcc = 0.7322 ± 0.0125 , and Regret = 0.0092 ± 0.0004 ; running memory slightly improved task accuracy to 0.5253 ± 0.0052 , but shuffled memory was nearly identical at 0.5250 ± 0.0048 .

Dataset	Best Note	Best Memory	Density	Note TaskAcc	Memory TaskAcc
Breast Cancer	JSON	running	0.50	.944 $\pm$.015	.947 $\pm$.012
FashionMNIST	compact	none	0.12	.783 $\pm$.004	.738 $\pm$.014
20 Newsgroups	JSON	running	0.20	.523 $\pm$.009	.525 $\pm$.005

Table 16: Reason-hunt stress test. Note format matters, but running memory is mostly neutral once shuffled and noisy controls are included.

Format diagnostics. Note format was a real factor on all three datasets, but with different

winners. Breast Cancer had a small format gap: best mean note accuracy 0.9386 versus worst 0.9337. FashionMNIST had the largest format gap: compact notes averaged 0.7547 versus the worst format at 0.7189. 20 Newsgroups had a moderate format gap: JSON averaged 0.4608 versus the worst format at 0.4507. This supports keeping notes structured and parseable, but not assuming one universal textual layout.

Memory diagnostics. Real running memory did not clearly beat controls. On Breast Cancer, running, shuffled, and noise memory were effectively tied, with memory-real gain versus shuffled equal to 0.0000. On FashionMNIST, running memory was slightly worse than shuffled memory and worse than no memory. On 20 Newsgroups, running memory beat shuffled memory by only 0.0002 mean task accuracy. These controls weaken the claim that the current memory state is learning useful temporal error patterns.

Density and oracle diagnostics. Density was the strongest reason on FashionMNIST and 20 Newsgroups. FashionMNIST had a density gain of 0.0616 and a large oracle gap of 0.1353, indicating controller underfit and useful expert diversity. 20 Newsgroups had an even larger density gain of 0.1248, while the reported oracle-task diagnostic was below the best single expert, suggesting that the cost-adjusted gate oracle was not the same as the task-accuracy oracle. Breast Cancer had high pairwise expert agreement (0.9370), a small oracle gap (0.0234), and only moderate density gain (0.0175), so its difficulty appears to be redundancy rather than lack of routing signal.

What it means. The reason hunt sharpens the next design target. Note format matters enough to standardize it, data density matters strongly for image and text routing, and the current memory mechanism is not yet a reliable source of improvement because shuffled and noisy controls perform similarly. The next controller should prioritize stronger candidate policies and explicit memory gating before claiming that past-mistake memory improves routing.

24 Router-Oracle Signal Stress-Test Summary

What we did. We ran a router/oracle stress test to answer two follow-up questions: when the router chooses the best cost-adjusted gate, and which signal source is actually useful for routing. The benchmark reused Breast Cancer, FashionMNIST, and 20 Newsgroups. For each dataset, it first searched note-only routers across four note formats—JSON, table, Q&A, and compact text—then compared seven signal families: note only, meta only, data only, note plus meta, note plus data, meta plus data, and all signals.

How we did it. Each sample had four expert predictions, one for each gate. The oracle gate was defined by true-class probability minus gate cost, so it measured cost-adjusted gate value rather than task accuracy alone. Note text encoded dataset, modality, pairwise agreement, mean confidence, margin, entropy, ranked candidate gates, and a best-hint gate. Meta features used the same underlying expert-output statistics directly, while data features used standardized tabular inputs, PCA image features, or TF-IDF text features. Routers were trained on gate labels and evaluated by task accuracy, gate accuracy, regret, best single expert accuracy, oracle task accuracy, and miss diagnostics.

What we got. On Breast Cancer, Q&A was the best note format by gate accuracy at full density, reaching $\text{TaskAcc} = 0.9415 \pm 0.0109$, $\text{GateAcc} = 0.9240 \pm 0.0041$, and $\text{Regret} = 0.0389 \pm 0.0059$. The best signal family was note plus data at full density, with $\text{TaskAcc} = 0.9649 \pm 0.0124$, $\text{GateAcc} = 0.8830 \pm 0.0477$, and $\text{Regret} = 0.0265 \pm 0.0076$; the best single expert still averaged 0.9737,

while the cost-adjusted oracle reached 0.9912. On FashionMNIST, Q&A was again the best note format at density 0.12, with TaskAcc = 0.7880 ± 0.0075 , GateAcc = 0.6938 ± 0.0093 , and Regret = 0.1502 ± 0.0071 . The best signal family was note plus meta, with TaskAcc = 0.7901 ± 0.0050 , GateAcc = 0.6936 ± 0.0116 , and Regret = 0.1492 ± 0.0067 , below the best single expert at 0.8002 and far below the oracle task accuracy of 0.9361. On 20 Newsgroups, JSON was the best note format at density 0.20, with TaskAcc = 0.5257 ± 0.0062 , GateAcc = 0.7320 ± 0.0045 , and Regret = 0.0086 ± 0.0002 . The best signal family by task accuracy was meta only, with TaskAcc = 0.5569 ± 0.0109 , GateAcc = 0.5429 ± 0.0287 , and Regret = 0.0418 ± 0.0052 .

Dataset	Best Note	Best Family	Density	TaskAcc	GateAcc	Regret
Breast Cancer	Q&A	note+data	1.00	.965 $\pm$.012	.883 $\pm$.048	.027 $\pm$.008
FashionMNIST	Q&A	note+meta	0.12	.790 $\pm$.005	.694 $\pm$.012	.149 $\pm$.007
20 Newsgroups	JSON	meta only	0.20	.557 $\pm$.011	.543 $\pm$.029	.042 $\pm$.005

Table 17: Router/oracle signal stress test. The best routing signal is dataset-dependent: tabular routing benefits from note text plus raw data features, image routing benefits from note text plus expert-output meta features, and text routing benefits most from expert-output meta features alone.

Oracle-miss diagnostics. Misses were strongly associated with uncertainty and expert disagreement on Breast Cancer and FashionMNIST. For Breast Cancer, the best-family router had an overall gate hit rate of 0.7836 and mean oracle regret 0.0475 across all tested densities and seeds; miss correlation was +0.2537 with low confidence, +0.2489 with high entropy, +0.2544 with low margin, and +0.3082 with expert disagreement. For FashionMNIST, the best-family router had gate hit rate 0.6744 and regret 0.1683; misses correlated +0.4510 with low confidence, +0.4605 with high entropy, +0.4501 with low margin, and +0.5544 with expert disagreement. For 20 Newsgroups, the best-family router had gate hit rate 0.6225 and regret 0.0275; uncertainty correlations remained moderate, but disagreement was weaker at +0.1318.

What it means. The test separates gate accuracy from task accuracy. On 20 Newsgroups, note-only routing had high gate accuracy and low regret but lower task accuracy than meta-only routing, because the cost-adjusted oracle is not identical to the task-accuracy oracle. On Breast Cancer, adding data features improved the router but still did not beat the best single expert on average, showing that high expert agreement leaves little room for routing gains. On FashionMNIST, the large gap between router accuracy and oracle task accuracy indicates useful expert diversity that the current controller has not captured. Overall, note format matters, but the underlying routing signal matters more: the next controller should learn when to use note text, expert-output meta features, and raw data features separately rather than assuming that one representation dominates.

25 Expanded Router Signal-Ablation Summary

What we did. We ran a longer router performance and signal-ablation study over the same three diagnostic datasets: Breast Cancer as a tabular task, FashionMNIST as an image task, and 20 Newsgroups as a text task. The study varied training fraction and seed, then compared note formats, signal families, and oracle-miss diagnostics. The goal was to determine whether routing success comes mainly from note text, note layout, expert-output meta-features, raw input features, or uncertainty and disagreement signals.

What we got. The broad pattern was that structured notes are useful routing objects, but their value depends on dataset and modality. JSON and table notes were the most consistently strong formats across datasets, compact notes were especially competitive on FashionMNIST, and Q&A notes were competitive but less consistently best. Across signal families, note-only routing was strongest or near-strongest in many settings, especially on tabular and text tasks. Meta-only and all-signal routers helped more on difficult image and text settings, while data-only routers sometimes performed surprisingly well, showing that raw input features can contain useful routing cues even without explicit note text.

Dataset	Note Fmt.	Signal Family	GateAcc	Miss Driver
Breast Cancer	JSON/table	note-only	.92-.95	residual disagreement
FashionMNIST	compact/JSON	meta/all	.65-.71	expert disagreement
20 Newsgroups	JSON	meta+data	.69-.75	disagreement/uncertainty

Table 18: Expanded router signal-ablation summary. Structured note formats are useful, but the best signal family changes by modality and difficulty.

Oracle-miss analysis. Across the expanded run, correct routing was associated with high router confidence, low entropy, and low expert disagreement. Routing misses were associated with the opposite pattern: lower confidence, higher entropy, and stronger disagreement among the four expert gates. Expert disagreement was the most consistent diagnostic of misses, with positive miss correlations in the reported range of roughly 0.44–0.82 depending on dataset and split. This makes disagreement a better next-step control signal than note format alone.

What it means. The expanded ablation is consistent with the paper’s central claim while keeping it conservative. Structured entry notes can carry decision-relevant routing information, and note-only routers can be strong. However, the best router should not rely only on text form. It should combine machine-readable notes with explicit uncertainty and disagreement features, and it should treat raw data features as optional routing evidence when they improve validation performance. The next controller should therefore test disagreement-aware routing as a first-class mechanism: when experts agree, use a simple low-cost policy; when experts disagree, allocate more budget to routing, memory, or a learned mixture.

26 Uncertainty-and-Memory Stress-Test Summary

What we did. We ran a second GLAMARC stress test focused on when memory helps. The benchmark asked whether running memory beats shuffled or noisy controls, whether memory helps more on hard or uncertain samples, and whether the gain changes with dataset and seed. It reused Breast Cancer, FashionMNIST, and 20 Newsgroups, with four note formats and four memory modes: none, running, shuffled, and noise.

How we did it. Each note encoded modality, pairwise agreement, mean top-1 confidence, mean margin, mean entropy, ranked gate candidates, and a best-hint gate. The best note format was selected by gate-set performance, then memory variants were evaluated on the same selected format. In addition to aggregate task accuracy and regret, the test computed per-sample deltas for running memory versus no memory, shuffled memory, and noise memory. These deltas were sliced by entropy, low confidence, disagreement proxy, and whether the best gate was Memory.

What we got. On Breast Cancer, JSON at density 0.50 was again the best note format, with

TaskAcc = 0.9444 ± 0.0149 , GateAcc = 0.9064 ± 0.0083 , and Regret = 0.0435 ± 0.0135 . Running memory at density 0.50 reached TaskAcc = 0.9474 ± 0.0124 , but shuffled and noise memory reached the same task accuracy, so the running memory gain was not specific to real history. On FashionMNIST, compact notes at density 0.12 were best, with TaskAcc = 0.7834 ± 0.0037 , while the best memory mode was no memory at 0.7383 ± 0.0140 ; running memory was worse at 0.7257 for the same density. On 20 Newsgroups, JSON at density 0.20 was best among notes, with TaskAcc = 0.5232 ± 0.0089 , while running memory reached 0.5253 ± 0.0052 and shuffled memory reached 0.5250 ± 0.0048 .

Dataset	Best Note	Best Memory	Density	Note TaskAcc	Memory TaskAcc
Breast Cancer	JSON	running	0.50	.944 $\pm$.015	.947 $\pm$.012
FashionMNIST	compact	none	0.12	.783 $\pm$.004	.738 $\pm$.014
20 Newsgroups	JSON	running	0.20	.523 $\pm$.009	.525 $\pm$.005

Table 19: Uncertainty-and-memory stress test. Running memory gives small aggregate gains on Breast Cancer and 20 Newsgroups, but does not clearly beat shuffled or noisy controls.

Memory-help slices. The per-sample analysis did not find strong evidence that memory helps specifically on hard or uncertain examples. On Breast Cancer, running memory had mean delta +0.0000 versus no memory and +0.0000 versus shuffled memory; the high Memory-needed slice was negative (-0.0303 versus no memory). On FashionMNIST, running memory hurt overall, with mean delta -0.0109 versus no memory, and was negative in most entropy and low-confidence slices. On 20 Newsgroups, running memory was slightly positive versus no memory (+0.0008) and nearly tied with shuffled and noise controls; the highest-entropy and lowest-confidence slices had small positive deltas around +0.003.

Correlation diagnostics. Correlations between memory benefit and uncertainty were weak. Breast Cancer had near-zero correlations with entropy (+0.0232), low confidence (-0.0401), and disagreement proxy (-0.0232). FashionMNIST also had near-zero correlations, despite overall negative memory impact. 20 Newsgroups showed only tiny positive correlations with entropy (+0.0093) and low confidence (+0.0084). Thus, the current memory representation does not reliably identify the uncertain cases where history should help.

What it means. This stress test narrows the claim further. Memory can produce small aggregate improvements, but those improvements are not yet clearly tied to real temporal history or to uncertainty-aware behavior. The next version should make memory access conditional: use memory only when uncertainty, disagreement, or recent regret predicts a real benefit, and require it to beat shuffled and noisy controls before claiming that past mistakes improve routing.

27 Calibration and Selective-Routing Closure Summary

What we did. We ran a final closure report for calibration and selective routing. This version re-ran the compact stress test, selected the best note format per run, selected the best signal family per dataset, and then computed calibration and selective-routing behavior for the selected family. The goal was to produce a final paper-ready check of whether router confidence can be trusted and whether high-confidence routing subsets are meaningfully better than full-coverage routing.

How we did it. Each dataset was evaluated across the same training fractions and seeds as the

earlier stress tests. For each split, the code trained four gate experts, generated notes in JSON, table, Q&A, and compact formats, trained note routers, and then compared signal families: note-only, meta-only, data-only, note-plus-meta, note-plus-data, meta-plus-data, and all-signals. At the dataset level, the best family was chosen by mean routed task accuracy. Calibration was measured by expected calibration error (ECE), and selective routing was measured by sweeping confidence thresholds from full coverage to threshold 0.95.

What we got. On Breast Cancer, the final selected best family was note-plus-data. Mean TaskAcc across run-level family evaluations was 0.9328 ± 0.0196 , GateAcc was 0.7995 ± 0.1693 , Regret was 0.0572 ± 0.0221 , best single expert accuracy was 0.9669 ± 0.0136 , and oracle task accuracy was 0.9903 ± 0.0077 . On FashionMNIST, the selected best family was note-plus-meta. Mean TaskAcc was 0.7471 ± 0.0270 , GateAcc was 0.6272 ± 0.0811 , Regret was 0.1862 ± 0.0237 , best single expert accuracy was 0.7932 ± 0.0092 , and oracle task accuracy was 0.9286 ± 0.0072 . On 20 Newsgroups, the selected best family was note-plus-data. Mean TaskAcc was 0.4564 ± 0.0554 , GateAcc was 0.6919 ± 0.0420 , Regret was 0.0126 ± 0.0062 , best single expert accuracy was 0.5574 ± 0.0541 , and oracle task accuracy was 0.5117 ± 0.0573 .

Dataset	Best Family	TaskAcc	GateAcc	Regret	ECE	Selective Gain
Breast Cancer	note+data	.933 $\pm$.020	.800 $\pm$.169	.057 $\pm$.022	.071	+.004
FashionMNIST	note+meta	.747 $\pm$.027	.627 $\pm$.081	.186 $\pm$.024	.087	+.152
20 Newsgroups	note+data	.456 $\pm$.055	.692 $\pm$.042	.013 $\pm$.006	.378	+.172

Table 20: Final calibration and selective-routing closure report. Selective gain is the task-accuracy increase at confidence threshold 0.95 relative to full coverage for the selected best family.

Note-format and family diagnostics. JSON remained the most common selected note format overall, but not universally: Breast Cancer split-level winners included JSON, Q&A, table, and compact; FashionMNIST used JSON most at low density and compact more often at higher density; 20 Newsgroups mostly selected JSON, with some Q&A and table wins. This weakens any claim that one note format is universally best. Signal-family selection was also dataset-dependent. The closure report selected note-plus-data for Breast Cancer and 20 Newsgroups, and note-plus-meta for FashionMNIST, while individual split winners varied among note-only, data-only, meta-only, all-signals, and combined families.

Calibration diagnostics. Breast Cancer was the best calibrated of the three closure runs, with ECE 0.0714, mean confidence 0.9820, and mean accuracy 0.9376 for the selected family. FashionMNIST was moderately calibrated, with ECE 0.0872, mean confidence 0.8393, and mean accuracy 0.7578. 20 Newsgroups was weakly calibrated and over-confident, with ECE 0.3776, mean confidence 0.8394, and mean accuracy 0.4619. The text setting therefore remains the clearest case where raw router confidence should not be treated as calibrated probability without additional calibration or fallback logic.

Selective-routing diagnostics. Selective routing produced only a small gain on Breast Cancer: at threshold 0.95, coverage was 0.960 and TaskAcc rose from 0.9376 to 0.9411. On FashionMNIST, selective routing was much more useful: at threshold 0.95, coverage was 0.439 and TaskAcc rose from 0.7578 to 0.9100, while GateAcc rose to 0.9189 and Regret fell to 0.0391. On 20 Newsgroups, selective routing also helped but required low coverage: at threshold 0.95, coverage was 0.321 and TaskAcc rose from 0.4619 to 0.6335, while GateAcc rose to 0.8583 and Regret fell to 0.0041.

What it means. The closure report makes the final design target sharper. Confidence is useful for

deciding when to trust the router, but calibration quality is dataset-dependent. Selective routing can greatly improve accuracy on hard image and text tasks, but it does so by abstaining from many examples. The next controller should therefore not force every example through one hard gate. It should choose among three actions: route directly when confidence is high, fall back to a safer mixture or rule prior when confidence is low, and invoke memory or a second-stage path when uncertainty and expert disagreement suggest that extra control may help.

28 MasterFabric Digits Smoke-Test Summary

What we did. We tested an early integration of the gate-logged controller with a previous “MasterFabric” architecture idea. The goal was not to replace the simpler routing benchmarks, but to check whether the four-gate interface can live inside a small end-to-end neural fabric with persistent memory, differentiable routing, audit notes, and a shared classifier. The test used sklearn digits as a lightweight smoke test.

How we did it. The model projected each digit into a hidden state and passed it through four neural experts: Compute, Route, Memory, and Control, where Control is the MasterFabric analogue of the Stop/Restart gate. Compute used a small feed-forward transformation; Route fused the hidden state with retrieved memory context; Memory used a persistent top- k memory buffer; and Control used a bounded trajectory-control transformation. A router selected among the four expert outputs using hard Gumbel-softmax during training and argmax at test time. The model also produced an entry audit vector before routing and an exit audit vector after routing, with an audit loss encouraging the two to align. A small “flashlight” head produced a burst probability that slightly scaled the selected latent state. During training, the router was supervised by the current oracle expert under the shared classifier, while the full loss combined task cross-entropy, gate supervision, audit loss, and an entropy bonus.

What we got. After 15 epochs, the training task loss fell from 1.9145 to 0.0514, the audit loss fell from 0.0615 to 0.0362, and the total loss fell from 2.4039 to 0.5230. On the held-out digits test split, the model reached model accuracy 0.9694 and routed task accuracy 0.9694. The current oracle task accuracy was 0.9917, gate accuracy was 0.6000, mean regret was 0.0198, and router entropy was 0.7105.

Model	Model Acc	TaskAcc	Oracle TaskAcc	GateAcc	Regret
MasterFabric digits smoke test	.969	.969	.992	.600	.020

Table 21: MasterFabric digits smoke test. The integrated neural fabric reaches strong task accuracy and low regret, but still leaves a clear gap to the oracle gate and only moderate gate accuracy.

What it means. This result is a useful bridge to the broader architecture idea. The four-gate interface can be embedded in a neural fabric with persistent memory and audit vectors, and this small version can train end-to-end on a real dataset. However, the result also repeats the paper’s main caution: high task accuracy does not mean the router has learned the oracle gate. The model reaches 0.9694 accuracy with low regret, but gate accuracy remains 0.6000 and the oracle task ceiling is 0.9917. The next MasterFabric version should therefore inherit the lessons from the simpler experiments: constrain the router with executable priors, learn budgeted or path-based gate use, and make memory reads conditional rather than always available.

29 Final Discussion

The main conclusion is now clear: the strongest and most reliable result in this paper is not unconstrained hard gate selection, but constrained gate control. Across the experiments, soft budget allocation, short ordered paths, and simple rule priors are consistently more stable than a one-step learned chooser. The synthetic controller shows that gate-logged auditing is trainable in principle: the system can learn to predict gate value, reduce regret, and align entry notes with observed outcomes. That is the cleanest positive result in the paper.

The real-data results sharpen the story rather than broaden it. On digits, the best performance comes from budgeted mixtures and short paths, not from unconstrained gate choice. In the Master-Fabric smoke test, the integrated neural fabric reaches strong task accuracy, but the gate accuracy is still only moderate, which shows that good final accuracy does not automatically mean good internal routing. The paper therefore supports auditability and control, but not the claim that learned hard gating is already solved.

The cross-dataset studies show that routing is strongly modality-dependent. When experts agree, routing has limited room to improve and simple no-memory or low-cost mixtures are often enough. When experts disagree, disagreement-aware mixtures, meta-features, or additional routing stages become useful. Note format matters, but it is secondary to the quality of the candidate experts and the quality of the routing signal. JSON, table, Q&A, and compact notes are all useful in some settings, but no single format dominates across all datasets.

The memory results are deliberately conservative. Running memory sometimes gives small gains, but those gains are often matched by shuffled or noisy controls. That means the current memory mechanism is not yet a reliable source of causal improvement. The safe interpretation is that memory can help sometimes, but only conditionally, and only when it beats explicit control baselines.

The final calibration and selective-routing results make the practical implication even sharper. Router confidence can identify higher-quality subsets, and selective routing can improve accuracy, especially on harder datasets. But confidence is not uniformly calibrated, and the improvement often comes from abstaining on uncertain cases rather than from better full-coverage routing. The right conclusion is therefore not that the router is always trustworthy, but that confidence is useful when it is calibrated and paired with a fallback policy.

Overall, the paper supports gate-logged control as an audit-and-routing framework, but its strongest evidence favors constrained control over unconstrained hard choice. The most defensible current claim is that internal decisions can be made measurable, trainable, and auditable, and that simple structured policies currently outperform a fully learned free-form router.

30 Limitations

Several limitations remain. First, most real-data experiments are small or diagnostic rather than large benchmark studies. Second, many results reuse lightweight expert pools, so gains may depend on the quality and diversity of those experts. Third, some oracles are cost-adjusted gate oracles rather than pure task-accuracy oracles, which can make gate accuracy and task accuracy disagree. Fourth, the paper reports many exploratory sweeps; the strongest findings should be re-run as pre-registered ablations before being treated as stable conclusions. Finally, the Master-

Fabric experiment is only a smoke test and should not be interpreted as evidence that the broader architecture is complete.

31 Future Work

The next version should move from unconstrained gate choice toward controlled decision structure. The strongest empirical pattern in this paper is that stable mixtures and short ordered paths are more reliable than a single hard gate. Future controllers should therefore predict not only a gate, but also a budget allocation, a short path, a memory-read decision, and a fallback or abstention decision.

A practical next design is a three-layer controller. The first layer is an executable rule prior: use low-cost or no-memory policies when experts agree, use disagreement-aware mixtures when experts diverge, and allow Memory only when uncertainty, disagreement, or recent regret suggests benefit. The second layer is a learned adjustment layer that fine-tunes the budget or path around that prior. The third layer is an audit layer that records predicted gain, predicted cost, selected budget, selected path, confidence, fallback decision, and memory-use reason, then scores those predictions after execution.

Future benchmarks should separate at least three targets: the task oracle, the cost-adjusted gate oracle, and the budgeted-mixture oracle. That separation would make it clearer whether a failure comes from choosing the wrong gate, optimizing the wrong objective, using weak experts, or trusting an uncalibrated router. Memory experiments should also always include no-memory, shuffled-memory, and noise-memory controls so that any claimed memory benefit is causal rather than incidental.

The next paper should therefore not try to prove that hard routing is universally best. It should instead show that routing can be stabilized by constraints, priors, calibration, and explicit controls. That would turn gate-logged control from a diagnostic framework into a practical orchestration layer.

32 Conclusion

We introduced a four-gate self-auditing controller for adaptive compute, routing, memory, and trajectory control. The central idea is to make internal decisions inspectable and trainable: before acting, the controller records what it expects a gate to do; after acting, it records what happened and compares the decision with alternatives.

The evidence supports a modest conclusion. Gate-logged control can learn cleanly in controlled settings and can sometimes improve real routing when constrained as a budget, path, or rule-guided policy. It is also useful as a diagnostic tool: it exposes when task accuracy, gate accuracy, regret, note format, memory, and confidence calibration disagree. The current results do not justify a broad claim that hard learned gate choice or running memory is generally sufficient. The most promising next step is a calibrated, disagreement-aware controller that learns stable budgets and paths, uses executable rule priors, and treats memory as a conditional intervention that should be tested against explicit controls.